5. Работа с шинами событий. Docker. RabbitMQ

Цель работы

Научиться уведомлять сторонние сервисы об изменениях в системе через брокер сообщений и добавлять поведение к существующему классу, не изменяя его код.

Материалы для подготовки

- Основные понятия RabbitMQ: https://www.rabbitmq.com/tutorials/amqp-concepts
- RabbitMQ и .NET: https://www.rabbitmq.com/tutorials/tutorial-one-dotnet
- Docker: https://docs.docker.com/get-started/
- Паттерн «Декоратор»: https://metanit.com/sharp/patterns/4.1.php

Теоретические сведения

Зачем нужна шина событий

Архитектуры веб-приложений часто строятся на событийной модели. Приложение сообщает, что в системе произошло некоторое действие, и об этом требуется уведомить другие сервисы: систему мониторинга качества, службу оформления доставки, аналитику.

Прямой вызов этих сервисов из кода приложения означает, что приложение знает о каждом из них и падает вместе с любым. Брокер сообщений разрывает эту связь: приложение отправляет событие брокеру и продолжает работу, а кто и когда прочитает событие, его не касается.

Устройство RabbitMQ

Сообщение отправляется не в очередь напрямую, а в точку обмена, которая называется exchange. Очередь связывается с точкой обмена привязкой, и правило привязки определяет, какие сообщения в эту очередь попадут.

```
приложение → exchange → привязка → queue → потребитель
```

Разделение позволяет добавить второго потребителя со своей очередью, не трогая отправителя.

Развёртывание брокера

RabbitMQ запускается из образа Docker. Образ с суффиксом `management` включает веб-интерфейс, через который видно очереди и накопленные в них сообщения.

```
docker run -d --name rabbitmq \
    -p 5672:5672 -p 15672:15672 \
    rabbitmq:3-management
```

Порт `5672` используется приложением, порт `15672` веб-интерфейсом. Интерфейс открывается по адресу `http://localhost:15672`, имя пользователя и пароль по умолчанию `guest`.

Формат события

Событие представляет собой сообщение формата JSON с типом события и полями изменившейся записи.

```
{
  "eventType": "CREATE",
  "car": {
    "firm": "Toyota",
    "model": "Camry",
    "year": 2018,
    "power": 178,
    "color": "Red",
    "price": 25000
  }
}
```

Типы событий соответствуют трём изменяющим операциям: `CREATE`, `UPDATE`, `DELETE`.

Публикация сообщения

Примеры этого раздела построены на другой предметной области: книгах и издательствах. Переложить их на автомобили и дилеров предстоит самостоятельно.

```
var factory = new ConnectionFactory { HostName = "localhost" };
using var connection = factory.CreateConnection();
using var channel = connection.CreateModel();

channel.ExchangeDeclare("books_events_exchange", ExchangeType.Fanout);
channel.QueueDeclare("books_events_queue", durable: true,
    exclusive: false, autoDelete: false);
channel.QueueBind("books_events_queue", "books_events_exchange", "");

var body = Encoding.UTF8.GetBytes(JsonSerializer.Serialize(payload));
channel.BasicPublish("books_events_exchange", "", null, body);
```

Объявление точки обмена и очереди выполняется приложением при старте. Повторное объявление с теми же параметрами ничего не меняет, поэтому его безопасно вызывать при каждом запуске.

Добавление событий существующему классу

Отправку событий не следует дописывать внутрь класса доступа к данным: у него появится вторая обязанность, а вместе с ней зависимость от брокера.

Паттерн «Декоратор» позволяет обернуть существующий класс другим, реализующим тот же интерфейс. Обёртка вызывает исходный объект, а затем добавляет своё поведение.

```
public class EventPublishingRepository : IBookRepository
{
    private readonly IBookRepository inner;
    private readonly IEventPublisher publisher;

    public Book Add(Book book)
```

```
    {
        var created = inner.Add(book);
        publisher.Publish("CREATE", created);
        return created;
    }
}
```

Исходный класс при этом не меняется ни на строку, а контроллер не знает, обёрнут он или нет: обёртка подставляется при настройке приложения.

> Событие отправляется после того, как операция с базой завершилась успешно. Событие, отправленное до записи, сообщит подписчикам об изменении, которого не произошло.

Задания

1. Разверните RabbitMQ из образа Docker и откройте его веб-интерфейс.
2. Опишите класс, отправляющий сообщение в точку обмена `cars_events_exchange`, связанную с очередью `cars_events_queue`.
3. Вынесите логику работы с данными из четвёртой работы в отдельный класс, если это не было сделано ранее.
4. Реализуйте обёртку над этим классом по паттерну «Декоратор», добавляющую отправку событий.
5. Обеспечьте отправку событий `CREATE`, `UPDATE` и `DELETE` при соответствующих операциях с автомобилями.
6. Выполните три запроса к API и покажите накопленные сообщения в веб-интерфейсе RabbitMQ.
7. Приведите в отчёте текст одного сообщения каждого типа.
8. Оформите отчёт по требованиям из общей части методических указаний.

Критерии приёмки

- Брокер развёрнут в контейнере, очередь и точка обмена названы согласно заданию.
- Сообщение имеет заданный формат, тип события соответствует выполненной операции.
- Отправка событий реализована обёрткой, исходный класс доступа к данным не изменён.
- Контроллер о существовании брокера не знает.
- Событие отправляется после успешного завершения операции с базой.
- Студент показывает сообщения в веб-интерфейсе брокера и поясняет путь сообщения от контроллера до очереди.

Контрольные вопросы

1. Зачем нужен брокер сообщений, если сервисы можно вызвать напрямую?

2. Что такое exchange и чем он отличается от очереди?
3. Что произойдёт с сообщением, если ни одна очередь не связана с точкой обмена?
4. Зачем объявлять очередь при каждом запуске приложения?
5. Что делает паттерн «Декоратор» и почему здесь он уместнее правки исходного класса?
6. Почему событие отправляется после операции с базой, а не до неё?
7. Что произойдёт с работой API, если брокер окажется недоступен, и каким вы считаете правильное поведение?